Investigating Populations and Predator-Prey Dynamics

Jacob Bonner, August Hauter, Ryan Sabin

APPM 2360

February 2019

1 Introduction

This report investigates the dynamics of predator-prey interactions through the application of differential equations. To study populations and predator-prey relationships, we consider an environment occupied by mountain lions and mule deer. We evaluate the two populations, as well as general predator-prey interdependence, by creating and interpreting three different mathematical models.

The simplest model is created by modeling each population individually using logistic differential equations of the form

$$\frac{dy}{dt} = r(1 - \frac{y}{L})y$$

A general explanation of this model is that the population in question will grow exponentially until there is not enough resources to maintain the current growth rate. Then the growth will continue but instead of exponential growth it will act under bounded growth as it approaches some carrying capacity L.

The next model is The Lotka-Volterra System, compromised of the two autonomous differential equations

$$\frac{dx}{dt} = a_x x - c_x xy$$
$$\frac{dy}{dt} = -a_y y + c_y xy$$

where x represents prey, and y the predators. This model is a result of the Balance Law, in which the net rate of change of a population is equal to the rate-in of members minus the rate-out of members. The interactions between the species are proportional to the product of the population sizes, which effect each population differently.

The final model analyzed in this report is The Logistic Predator-Prey System, where x and y hold the same meaning as before. This model is similar to the previous system, only with a modified term in the prey population.

$$\frac{dx}{dt} = a_x x(1 - \frac{x}{L_x}) - c_x xy$$
$$\frac{dy}{dt} = -a_y y + c_y xy$$

The natural behavior of such a system is that in the absence of predation, the prey population will obey a logistic growth model instead of an exponential growth model.

2 Modeling Individual Populations

One way to mathematically study the populations of mountain lions and deer is by individually modeling each population with logistic equations. To simplify the system, we assume that mountain lions are protected from hunting and have no natural predators. However, the mountain lions depend on the deer for food, and since the deer population is finite, the model necessitates a size constraint. As the two populations are modeled separately, this constraint is accounted for without directly including interactions with the deer. Therefore, the model representing the mountain lion population is the logistic equation

$$\frac{dx}{dt} = r(1 - \frac{x}{L})x \tag{1}$$

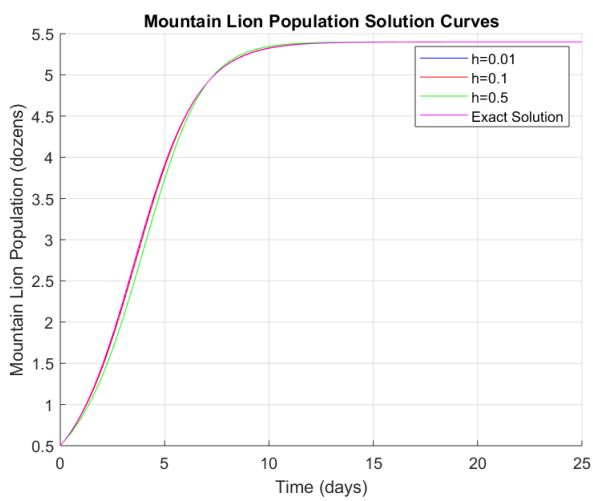
where $x(t)$ is the size of the population in dozens of animals, and t is the time in days. The positive parameter r is the initial growth rate, in $days^{-1}$, and L is the carrying capacity of the population, in dozens of animals (mountain lions or mule deer).

We can analyze the mountain lion population over time by first setting (1) equal to zero, and calculating that the equilibrium solutions to this equation are $x = 0$ and $x = L$. Additionally, we find the exact solution to the differential equation with unknown initial population $x(0) = x_0$

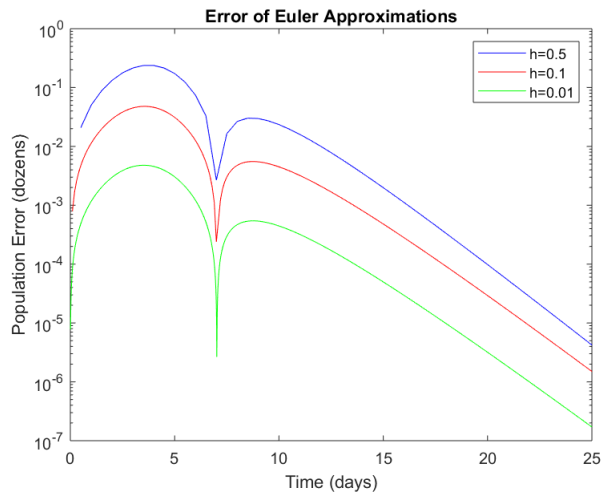
$$x(t) = \frac{L}{1 + (\frac{L}{x_0} - 1)e^{-rt}}$$

by using separation of variables (Appendix 1).

To qualitatively analyze the behavior of (1), we assume an initial population of 6 mountain lions (half a dozen), and parameters $r = 0.65$ and $L_M = 5.4$. Using Euler's Method over the interval $0 \leq t \leq 25$, we numerically solve the logistic equation with step sizes of $h_1 = 0.5$, $h_2 = 0.1$, and $h_3 = 0.01$.



Though difficult to see from the figure, as the step size h decreases, the accuracy of the approximation in comparison to the exact solution increases. To further show how step size affects accuracy, we compute the absolute error of each numerical approximation, which is defined as | Exact Solution - Approximate Solution |.



From the plot of the absolute error, we see that the smallest step size $h_3 = 0.01$ has the smallest error, and that an increase in step size consequently results in larger error. At $t = 7$ days, each of the error curves demonstrates a spiking behavior. This arises from the way that the Euler Approximations intersect the exact solution curve. Looking at the first figure, all of the numerical approximation curves intersect the exact curve around $t = 7$, resulting in the apparent behavior of the error. As time continues, the error gradually decreases as the population approaches carrying capacity. This is due to the Euler approximation using the solution's derivative, which funnels nearby values to that carrying capacity.

When a problem is solved numerically, it is important to make sure that the calculation carefully balances numerical "accuracy" – the correctness of the approximation – with numerical "efficiency," which is the amount of time to complete the calculation. To determine the accuracy and efficiency of our approximations, we carefully examine each step size.

Step size $h_1 = 0.5$ is extremely efficient compared to the others, because fewer steps are need to obtain a result. However, because of this large step size, the error is much larger, and therefore inaccurate results are obtained. On the other hand, step size $h_3 = 0.01$ is very accurate and closely resembles the exact solution, therefore having the greatest numerical accuracy. Despite the accuracy, having a step size of 0.01 is inefficient since it takes so many steps to obtain a full result. In the middle of h_1 and h_3 is h_2, at a value of 0.1. This step size makes an error that is greater than h_3, but less than h_1. In terms of efficiency, h_2 is again between the other two step sizes. Overall, h_2 is exceptionally efficient while simultaneously obtaining a relatively accurate approximation.

In a similar manner to the mountain lions, deer are limited by their resources, so it is appropriate to model the population logistically as well. However, mountain lions are a natural predator of deer, making them an additional means of restraining the deer population. To account for this limiting factor, we modify the logistic equation to include a predation term, or "harvesting term", that represents the amount of deer killed by mountain lions. Thus, the logistic equation for the population of deer becomes

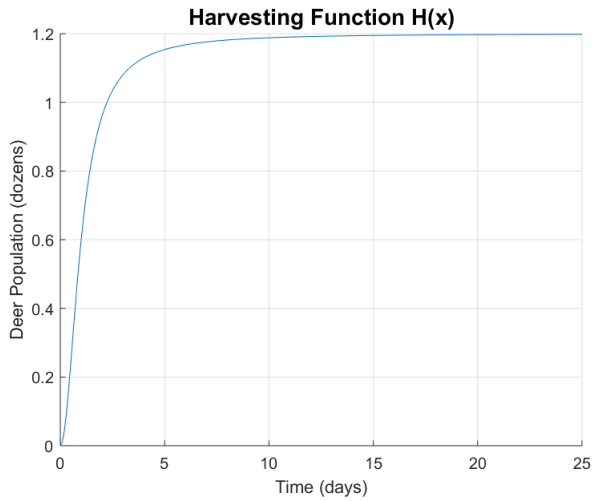
$$\frac{dx}{dt} = r(1 - \frac{x}{L})x - H(x) \tag{2}$$

where $x(t)$, t, r, and L hold the same meaning as in (1). The harvesting function used in this model is represented as

$$H(x) = \frac{px^2}{q + x^2} \tag{3}$$

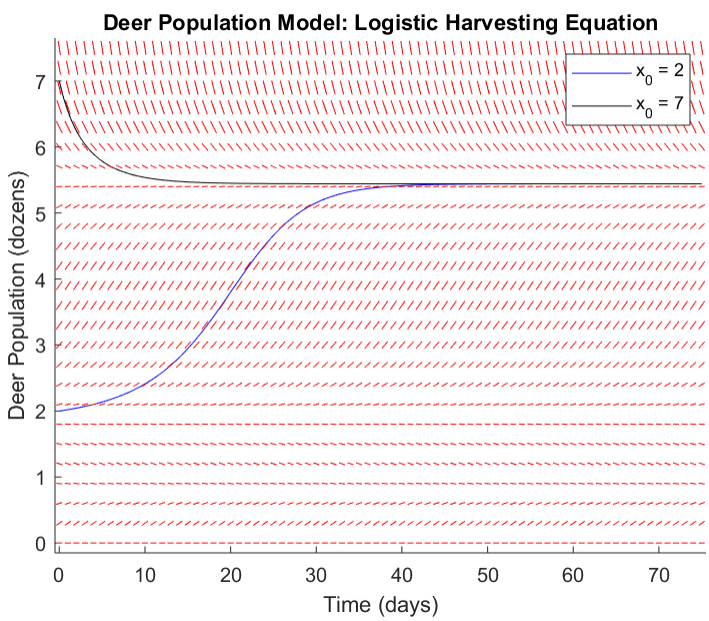
with parameters p and q indicating the skill of the mountain lions.

After evaluating (2) and (3), we see that the deer population model is a first-order nonlinear autonomous differential equation. The physical meaning of autonomy in this context is that the rate at which the deer population changes depends on external limitations, such as the environment and predation, but is not influenced by time. In other words, for any given number of deer, the population will always decrease or increase at the same rate for all time. In regards to the harvesting term, as x tends toward infinity, the function approaches the value p, and similarly as x tends toward zero, the function approaches zero.



This makes sense physically because as the deer population grows, mountain lions can find prey more easily. Likewise, if the deer population is diminished, mountain lions would have less prey in their environment, so the impact of the predation term is much less.

To visualize the behavior of (2), let the parameters take on the following values: $r = 0.65$, $L_D = 8.1$, $p = 1.2$, $q = 1$. Using Euler's Method with a step size of $h = 0.1$ on the interval $0 \leq t \leq 75$, we find two numerical approximations. Starting from initial deer populations of 2 dozen and 7 dozen, we plot the Euler approximations on top of the direction field for equation (2).



This plot shows two distinct population curves over time. The first, with an initial population of 2 dozen, follows the behavior of logistic growth. The other curve, with an initial population of 7 dozen deer, takes the shape of decay, or a decrease in size. These stark differences are due to the general behavior of a logistic equation, in which all nonzero initial values lead to solutions that approach the carrying capacity (L) asymptotically, from above and below. Therefore, the population experiencing the decay could be under strain, due to lack of resources in the habitat and increased ease of predation, until it nears the carrying capacity where it achieves a more stable population.

3 Modeling Population Interactions

3.1 The Lotka-Volterra System

We saw in the individual population models that direct species interactions are not accounted for. To more formally study how the populations respond to the pressures of both internal competition and external interactions, we utilize The Lotka-Volterra System

$$\frac{dx_1}{dt} = -ax_1 + bx_1x_2$$

$$\frac{dx_2}{dt} = cx_2 - dx_1x_2 \tag{4}$$

where the variable x_1 represents the predator population and x_2 represents the prey population. The cross terms, bx_1x_2 and dx_1x_2, indicate the interactions of the two species, which positively affect the predator and negatively affect the prey. In other words, a multitude of prey increases the predator's growth rate, but the prey's growth rate diminishes as a result of more predators. The positive parameters a, b, c, and d can be interpreted as follows:

a	predator mortality rate
b	predator attack rate / conversion efficiency (food into offspring)
c	prey growth rate
d	prey mortality rate / predator attack rate

Upon investigation, we see that the system is composed of two first-order nonlinear differential equations. The reason for the system being nonlinear is the second term in each equation, bx_1x_2 and dx_1x_2. Linear functions have a constant slope, but these cross terms cause each equation to vary as x or y changes, resulting in a slope that varies between points. The system is autonomous because each equation depends explicitly on x_1 and x_2 and only implicitly on the underlying variable t.

We can examine the behavior of this system by finding the nullclines and equilibrium solutions, and then plotting these along with the system's phase portrait. Nullclines are found by restricting one direction of motion in the system (Appendix 1).

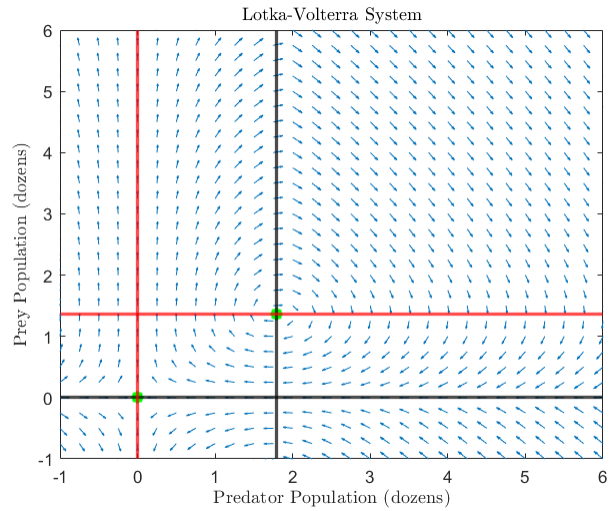
$$\frac{dx_1}{dt} = 0 \quad \Longrightarrow \quad x_1 = 0 \quad x_2 = \frac{a}{b}$$

$$\frac{dx_2}{dt} = 0 \quad \Longrightarrow \quad x_2 = 0 \quad x_1 = \frac{c}{d}$$

The horizontal nullclines are $x_1 = 0$ and $x_2 = \frac{a}{b}$, and the vertical nullclines are $x_2 = 0$ and $x_1 = \frac{c}{d}$. The equilibrium solutions are then found by the intersections of the horizontal and vertical nullclines (Appendix 1)

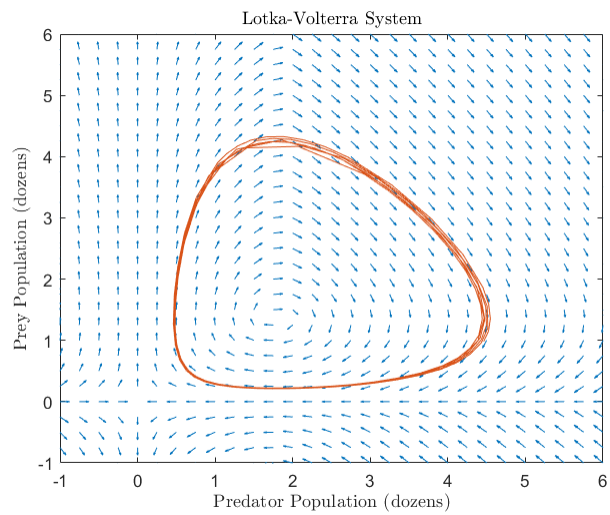
$$(x_1, x_2) = (0, 0) \quad (x_1, x_2) = (\frac{c}{d}, \frac{a}{b})$$

To visualize system (4), we assign parameter values $a = 1.5$, $b = 1.1$, $c = 2.5$, and $d = 1.4$, and plot the phase portrait. Additionally, we plot the horizontal nullclines in red, the vertical nullclines in black, and the equilibrium solutions in green.



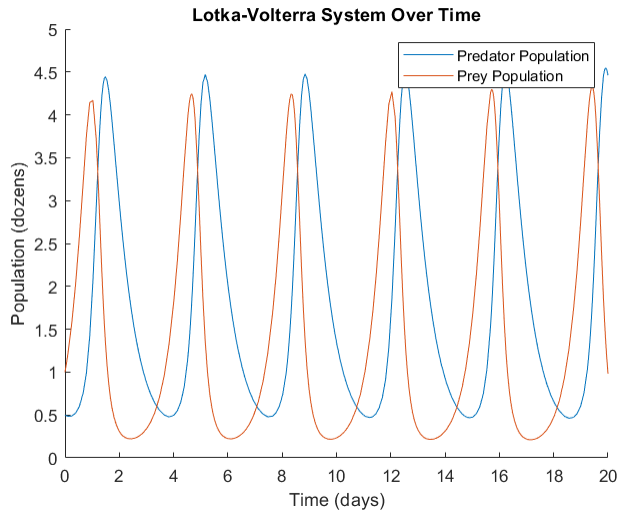
It can be seen that around the equilibrium solution at the point $(0, 0)$, the solutions are repelled from the point, meaning that $(0, 0)$ is unstable. The other equilibrium solution in the first quadrant shows the formation of closed loops around the point, which signifies a stable equilibrium solution. By interpreting both equilibrium solutions, we see that the one at the origin represents extinction for both species, and the other demonstrates balance between the predator and prey in a constant cycle.

To further show the idea of circulation, we plot the phase plane solution on the interval $0 \leq t \leq 20$ starting at the initial point $(x_1, x_2) = (0.5, 1.0)$.



The formation of the closed loops behaves as we expected, and makes sense when considered realistically. As prey population grows, predators have more food to eat and they in turn grow as well. When predator population gets too high and prey population diminishes significantly, predators run out of food and also drop. This is a cycle of growth and decay that is shown in the circular path traced out by the curve.

To further analyze the behavior of system (4), we use Matlab's solver `ode45` to simulate solutions from the initial condition $(x_1, x_2) = (0.5, 1)$. We plot the component curves $x_1(t)$ and $x_2(t)$ together against t on the interval $0 \leq t \leq 20$.



The large spikes in prey population are followed by large spikes in predator population, and a drop in prey again. After the drop in the number of prey, predator populations drop as well. This, again, examines the cyclic nature of the species populations. The combined spikes of population growth are said to be "in phase", meaning that the maximum of each population falls at or near the same value repeatedly over time.

3.2 The Logistic Predator-Prey System

The assumption of (4) that both species will experience exponential growth in the absence of inter-species interactions ignores the natural limits inflicted on prey by its environment. To further improve the practicality of species interaction, we modify the equations from (4) in such a way that incorporates the previously disregarded natural limits. The implementation of these adjustments gives rise to the Logistic Predator-Prey System, represented by

$$\frac{dx_1}{dt} = -ax_1 + bx_1x_2$$

$$\frac{dx_2}{dt} = c(1 - kx_2)x_2 - dx_1x_2 \tag{5}$$

in which x_1 and x_2 carry the same meaning as system (4). In the absence of predation, the prey population will obey a logistic growth model instead of exponential growth. In other words, if the prey population is small, the rate of growth is proportional to its size, and if the population is too large to be sustained by the environment, the population will decrease.

We analytically examine the behavior of this system by finding the nullclines and equilibrium solutions. Again, nullclines are found by restricting one direction of motion in the system (Appendix 1).

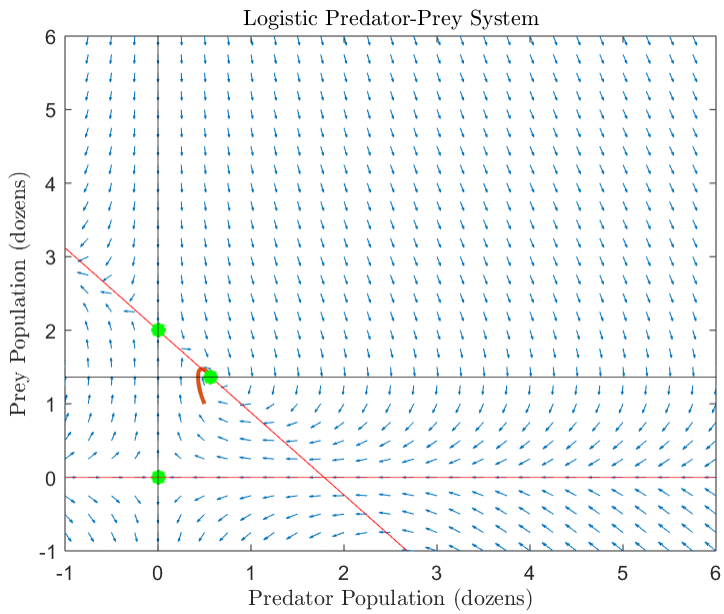
$$\frac{dx_1}{dt} = 0 \quad \Longrightarrow \quad x_1 = 0 \quad x_2 = \frac{a}{b}$$

$$\frac{dx_2}{dt} = 0 \quad \Longrightarrow \quad x_2 = 0 \quad x_2 = \frac{1}{k}(1 - \frac{d}{c}x_1)$$

The horizontal nullclines are $x_1 = 0$ and $x_2 = \frac{a}{b}$, and the vertical nullclines are $x_2 = 0$ and $x_2 = \frac{1}{k}(1 - \frac{d}{c}x_1)$. The equilibrium solutions are then found by the intersections of the horizontal and vertical nullclines (Appendix 1).

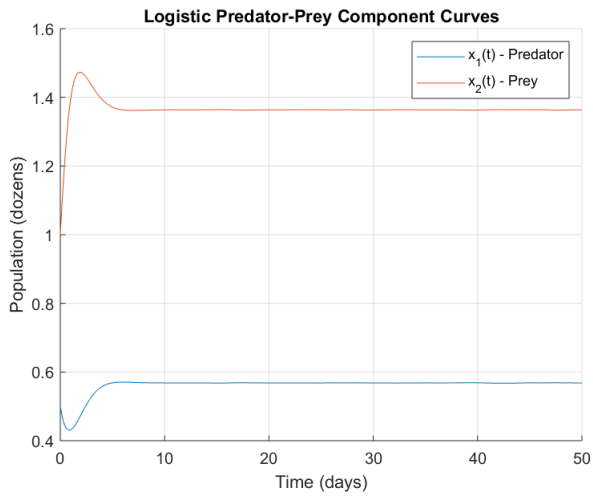
$$(x_1, x_2) = (0, 0) \quad (x_1, x_2) = (0, \frac{1}{k}) \quad (x_1, x_2) = (\frac{c}{d}(1 - k(\frac{a}{b})), \frac{a}{b})$$

In addition, we calculate the phase plane solution from an initial condition of $(x_1, x_2) = (0.5, 1)$, by plotting x_1 against x_2 in the x_1-x_2 plane. The plot containing the phase portrait, nullclines, equilibrium solutions, and the phase plane solution is shown below.



From the phase plane, we see the behavior of solutions and the stability of the equilibria. In particular, the phase plane solution converges into the equilibrium solution at $(x_1, x_2) = (\frac{c}{d}(1 - k(\frac{a}{b})), \frac{a}{b})$, meaning that it is stable. The stability of this point causes all solution curves with initial conditions in the first quadrant to be attracted to it. On the other hand, the two equilibrium solutions on the line $x_1 = 0$ are both unstable, which can be seen from how the phase portrait repels nearby solutions at those points.

We further investigate the behavior of the logistic system through means of plotting the component curves. Using Matlab's solver `ode45`, we simulate solutions starting from the initial condition $(x_1, x_2) = (0.5, 1)$, and we plot the component curves over the interval $0 \leq t \leq 50$.



As shown in the plot, we see that the component curves represent asymptotic behavior over time. Initially, the prey population has a large increase while the predator population has a slight drop. Then around $t = 3$ days, the prey population begins to diminish quickly, and as a result of predation, the predator population increases greatly. After this initial behavior, starting at $t = 5$ both populations smooth out and approach different values asymptotically.

3.3 Model Comparison

We have seen that The Lotka-Volterra System (4) is a very simple way to evaluate the basic interactions of predator and prey. With four parameters outlining the characteristics and basic interactions of each population, the system is able to accurately depict how species interactions negatively affect prey and positively affect predators. However, as discussed above, because it is a very simple model, important limiting factors are left unaccounted for. In particular, it ignores the natural limits imposed on a prey population by its environment.

Regarding the equation for prey population in (4), if predators were not in the system - in other words if the cross term was not present - prey population would grow exponentially. This does not make sense realistically, because over time prey populations would reach a capacity at which the resources of the environment are not sufficient. This is where the Logistic Predator-Prey System is superior.

Model (5) shows that in the absence of interactions, the prey population will not simply skyrocket; the population will follow logistic growth for a single species as described above. This makes the Logistic Predator-Prey model more realistic and, therefore, a better representation of predator-prey dynamics.

While it stands as our best model for examining populations, the logistic system can be modified to account for additional factors. Although it does take into account the impact from an environment, it does not take into account other obstacles a species may be affected by in various ecosystems. These obstacles may include droughts, deforestation, natural disasters, human hunting, or disease. To include these additional obstacles, extra parameters and interactive terms could be included in the model that would show how populations are affected.

Also, these models assume that mating and predation are done at a constant rate throughout the year, which is simply not realistic. Another addition that would make (4) and (5) more accurate in this regard is to include mating or hunting cycles. Many animals mate only once a year in a specific season, causing a large spike in populations during that time. There are also animals that hibernate or store food for certain amounts of time, so there would be a large drop in predation, and therefore an increase in prey populations during these times.

4 Conclusion

Upon investigating the interactions between deer and mountain lion populations, it becomes apparent that certain models more formally depict how populations fully respond to their environment. Specifically, when modeling populations individually using logistic equations, other than the harvesting term (3) used to describe the effect of predation on the deer population in (2), neither of the logistic equations (1) and (2) directly account for inter-species interactions. Thus, it is both the least realistic and most inaccurate method of the three we employed.

To include the interdependence of the two species, we moved on to the Lotka-Volterra model (4). In this system, each equation consists of cross terms to account for interactions, which we showed negatively affect the prey population and positively affect the predator population. However, we also proved that it lacks other important environmental factors.

In order to further the practicality of predator-prey interactions, we created our most complicated and realistic model - The Logistic Predator-Prey System (5) - in which we adjusted the prey equation from (4) to account for the impact of the species' environment. By including this new term, we saw that in the absence of predation, the prey population now obeys a logistic growth model instead of an exponential growth model, making (5) the most accurate approach.

Through these three mathematical models, we discovered that individually modeling predator and prey populations is inaccurate, and that using modified systems of differential equations better portrays realistic dynamics of predator-prey interactions.

Appendix 1

Calculations for Section 2

Finding Equilibrium Solutions of (1)

$$\frac{dx}{dt} = r(1 - \frac{x}{L})x$$
$$0 = rx(1 - \frac{x}{L})$$
$$rx = 0 \quad 1 - \frac{x}{L} = 0$$
$$x = 0 \quad 1 = \frac{x}{L}$$
$$x = 0 \quad x = L$$

Using Separation of Variables to Find General Solution of (1)

$$\frac{dx}{dt} = r(1 - \frac{x}{L})x$$
$$\frac{1}{x(1 - \frac{x}{L})}dx = rdt$$
$$\frac{1}{\frac{x}{L}(L - x)}dx = rdt$$
$$\int \frac{L}{x(L - x)}dx = \int rdt$$
$$L\int \left[\frac{1/L}{x} + \frac{1/L}{L - x}\right] dx = rt + C$$
$$\int \left[\frac{1}{x} + \frac{1}{L - x})\right] dx = rt + C$$
$$\ln \mid x \mid - \ln \mid L - x \mid = rt + C$$
$$\ln \mid \frac{x}{L - x} \mid = rt + C$$
$$\frac{x}{L - x} = Ce^{rt}$$
$$x = Ce^{rt}(L - x)$$
$$x = LCe^{rt} - xCe^{rt}$$
$$x + xCe^{rt} = LCe^{rt}$$
$$x(1 + Ce^{rt}) = CLe^{rt}$$
$$x = \frac{CLe^{rt}}{1 + Ce^{rt}}$$
$$x(t) = \frac{L}{1 + Ce^{-rt}}$$

Finding the Exact Solution to (1) Using Initial Population x_0

$$x(0) = \frac{L}{1 + Ce^0}$$
$$x_0 = \frac{L}{1 + C}$$
$$1 + C = \frac{L}{x_0}$$
$$C = \frac{L}{x_0} - 1$$
$$x(t) = \frac{L}{1 + (\frac{L}{x_0} - 1)e^{-rt}}$$

Calculations for Section 3.1

Analytically Finding the Nullclines and Equilibrium Solutions of System (4)

Horizontal Nullclines $(\frac{dx_1}{dt} = 0)$

$$0 = -ax_1 + bx_1x_2$$
$$0 = x_1(-a + bx_2)$$
$$x_1 = 0 \quad a = bx_2$$
$$x_1 = 0 \quad x_2 = \frac{a}{b}$$

Vertical Nullclines $(\frac{dx_2}{dt} = 0)$

$$0 = cx_2 - dx_1x_2$$
$$0 = x_2(c - dx_1)$$
$$x_2 = 0 \quad dx_1 = c$$
$$x_2 = 0 \quad x_1 = \frac{c}{d}$$

$$\textbf{Equilibrium Solutions } (\frac{dx_1}{dt} = 0 \text{ and } \frac{dx_2}{dt} = 0)$$

$$\text{Sub } x_1 = 0 \text{ into } \frac{dx_2}{dt} = 0$$

$$0 = cx_2 - d(0)x_2$$

$$0 = cx_2$$

$$x_2 = 0$$

$$\text{Sub } x_2 = \frac{a}{b} \text{ into } \frac{dx_2}{dt} = 0$$

$$0 = c(\frac{a}{b}) - dx_1(\frac{a}{b})$$

$$0 = (\frac{a}{b})(c - dx_1)$$

$$0 = c - dx_1$$

$$x_1 = \frac{c}{d}$$

$$(x_1, x_2) = (0, 0) \quad (x_1, x_2) = (\frac{c}{d}, \frac{a}{b})$$

Calculations for Section 3.2

Analytically Finding the Nullclines and Equilibrium Solutions of System (5)

$$\textbf{Horizontal Nullclines } (\frac{dx_1}{dt} = 0)$$

$$0 = -ax_1 + bx_1x_2$$

$$0 = x_1(-a + bx_2)$$

$$x_1 = 0 \quad a = bx_2$$

$$x_1 = 0 \quad x_2 = \frac{a}{b}$$

$$\textbf{Vertical Nullclines } (\frac{dx_2}{dt} = 0)$$

$$0 = c(1 - kx_2)x_2 - dx_1x_2$$

$$0 = x_2(c(1 - kx_2) - dx_1)$$

$$x_2 = 0 \quad 0 = c(1 - kx_2) - dx_1$$

$$x_2 = 0 \quad dx_1 = c(1 - kx_2)$$

$$x_2 = 0 \quad \frac{d}{c}x_1 = 1 - kx_2$$

$$x_2 = 0 \quad x_2 = \frac{1}{k}(1 - \frac{d}{c}x_1)$$

$$\textbf{Equilibrium Solutions } (\frac{dx_1}{dt} = 0 \text{ and } \frac{dx_2}{dt} = 0)$$

$$\text{Sub } x_1 = 0 \text{ into } \frac{dx_2}{dt} = 0$$

$$0 = c(1 - kx_2)x_2 - d(0)x_2$$

$$0 = cx_2 \quad 0 = 1 - kx_2$$

$$x_2 = 0 \quad x_2 = \frac{1}{k}$$

$$\text{Sub } x_2 = \frac{a}{b} \text{ into } \frac{dx_2}{dt} = 0$$

$$0 = \frac{a}{b}(c(1 - k(\frac{a}{b})) - dx_1)$$

$$0 = c(1 - k(\frac{a}{b})) - dx_1$$

$$x_1 = \frac{c}{d}(1 - k(\frac{a}{b}))$$

$$(x_1, x_2) = (0, 0) \quad (x_1, x_2) = (0, \frac{1}{k}) \quad (x_1, x_2) = (\frac{c}{d}(1 - k(\frac{a}{b})), \frac{a}{b})$$

Appendix 2

Created and Implemented Matlab Functions

Function 1 - Calculates Euler Approximations for Equation (1)

```
function y = MountainLionEuler (t,y0,h,rm,Lm)

n = length(t);
for i = 1:n
    if i == 1
        y(i) = y0;
    else
        y(i) = (rm*(1-((y(i-1)/Lm)))*y(i-1))*h + y(i-1);
    end
end
end
```

Function 2 - Calculates Solution Curve to Equation (1)

```
function y = MountainLionReal (t,y0,rm,Lm)
for i = 1:length(t)
y(i) = (Lm/(1+(((Lm/y0)-1)*exp(-rm*t(i)))));
end
```

Function 3 - Calculates the Absolute Error of the Euler Approximations to Equation (1)

```
function y = EulerError (x1,x2)
x3 = x2-x1;
y = abs(x3);
end
```

Function 4 - Calculates the Euler Approximations for Equation (2)

```
function [x] = eulersDeerPopulation(t,f,h,initialPop)
x = zeros(length(t),1);
x(1) = initialPop;
for n = 1:length(t)-1
    x(n+1) = x(n) + h*f(t(n),x(n));
end
end
```

Function 5 - Modified dirfield.m to Produce Direction Field of Equation (2)

```
function dirfield(f,tval,xval,plot_title)

[tm,xm]=meshgrid(tval,xval);
dt = tval(2) - tval(1);
dx = xval(2) - xval(1);
fv = vectorize(f);
if isa(f,'function_handle')
  fv = eval(fv);
end
xp=feval(fv,tm,xm);
s = 1./max(1/dt,abs(xp)./dx)*0.35;
h = ishold;
quiver(tval,xval,s,s.*xp,0,'.r'); hold on;
quiver(tval,xval,-s,-s.*xp,0,'.r');
if h
  hold on
else
  hold off
end
axis([tval(1)-dt/2,tval(end)+dt/2,xval(1)-dx/2,xval(end)+dx/2])

title(plot_title);
xlabel('Time (days)');
ylabel('Deer Population (dozens)');
end
```

Function 6 - Solves The Lotka-Volterra System

```
function vprime = solutions(t,v);
    a = 1.5;
    b = 1.1;
    c = 2.5;
    d = 1.4;
x1 = v(1); x2= v(2);
vprime = zeros(2,1);
vprime(1) = -a.*x1 + b.*x1.*x2;
vprime(2) = c.*x2 - d.*x1.*x2;
end
```

Function 7 - Solves the Logistic Predator-Prey System

```
function v_out = logistic_solver(t,v)
x1 = v(1);
x2 = v(2);
a = 1.5;
b = 1.1;
c = 2.5;
d = 1.4;
k = 0.5;
v_out = zeros(2,1);
v_out(1) = -a.*x1 + b.*x1.*x2;
v_out(2) = c.*(1-k.*x2).*x2 - d.*x1.*x2;
end
```

Matlab Code for Section 2

Plotting Euler Approximations and Solution Curve (See Function 1 and 2)

```
rm = 0.65;
Lm = 5.4;
h1 = 0.5;
h2 = 0.1;
h3 = 0.01;

t1 = 0:h1:25;
t2 = 0:h2:25;
t3 = 0:h3:25;

y1 = MountainLionEuler(t1,.5,h1,rm,Lm);
y2 = MountainLionEuler(t2,.5,h2,rm,Lm);
y3 = MountainLionEuler(t3,.5,h3,rm,Lm);
yr1 = MountainLionReal(t1,.5,rm,Lm);
yr2 = MountainLionReal(t2,.5,rm,Lm);
yr3 = MountainLionReal(t3,.5,rm,Lm);
```

```
figure
hold on
grid on
plot(t3,y3,'b')
plot(t2,y2,'r')
plot(t1,y1,'g')
plot(t3,yr3,'m')
legend('h=0.01','h=0.1','h=0.5','Exact Solution')
xlabel('Time (days)')
ylabel('Mountain Lion Population (dozens)')
title('Mountain Lion Population Solution Curves')
hold off

E1 = EulerError(y1,yr1);
E2 = EulerError(y2,yr2);
E3 = EulerError(y3,yr3);

semilogy(t1,E1,'b',t2,E2,'r',t3,E3,'g')
legend('h=0.5','h=0.1','h=0.01')
title('Error of Euler Approximations')
xlabel('Time (days)')
ylabel('Population Error (dozens)')
```

Calculating and Plotting the Absolute Error (See Function 3)

```
E1 = EulerError(y1,yr1);
E2 = EulerError(y2,yr2);
E3 = EulerError(y3,yr3);

figure
semilogy(t1,E1,'b',t2,E2,'r',t3,E3,'g')
legend('h=0.5','h=0.1','h=0.01')
title('Error of Euler Approximations')
xlabel('Time (days)')
ylabel('Population Error (dozens)')
```

Analyzing Harvesting Function for Deer Population

```
x = 0:0.1:25;
H = (1.2*x.^2)./(1 + x.^2);
figure
hold on
grid on
plot(x,H);
title('Harvesting Function H(x)','FontSize',14);
xlabel('Time (days)');
ylabel('Deer population (dozens)');
```

Plotting Euler Approximations and Direction Field (See Function 4 and 5)

```
f = @(t,x)(0.65).*(1-(x./8.1)).*x - (1.2*x.^2)./(1+x.^2);
h = 0.1;
t1 = 0:h:75;

p1 = eulersDeerPopulation(t1,f,h,2);
p2 = eulersDeerPopulation(t1,f,h,7);
f1 = @(t,x)(0.65)*(1-(x/8.1))*x - (1.2*x^2)/(1+x^2);

hold on
plot(t1,p1,'b')
plot(t1,p2,'k')
dirfield(f1,0:75,0:0.3:7.5,'Deer Population Model: Logistic Harvesting Equation');
legend('x_0 = 2','x_0 = 7');
hold off
```

Matlab Code for Section 3.1

Plotting Phase Portrait, Nullclines, and Equilibrium Solutions (Modified flow.m)

```
x1min = -1; x1max = 6; x2min = -1; x2max = 6;

x1step = .25; x2step = .25;

[x1, x2] = meshgrid(x1min:x1step:x1max, x2min:x2step:x2max);

a = 1.5;
b = 1.1;
c = 2.5;
d = 1.4;

dx1 = -a.*x1 + b.*x1.*x2;
dx2 = c.*x2 - d.*x1.*x2;

dx1 = dx1./sqrt(dx1.^2 + dx2.^2);
dx2 = dx2./sqrt(dx1.^2 + dx2.^2);

quiver(x1, x2, dx1,dx2,'AutoScaleFactor',0.5)

axis([x1min x1max x2min x2max])

xlabel('Predator Population (dozens)','Interpreter','latex')
ylabel('Prey Population (dozens)','Interpreter','latex')
title('Lotka-Volterra System','Interpreter','latex')

hold on
yline(0,'k','LineWidth',2);
xline(c/d,'k','LineWidth',2);
xline(0,'r','LineWidth',2);
yline(a/b,'r','LineWidth',2);
plot(0,0,'g*','LineWidth',5)
plot(c/d,a/b,'g*','LineWidth',5)
```

Plotting Phase Plane Solution Over Phase Portrait (Modified flow.m, See Function 6)

```
x1min = -1; x1max = 6; x2min = -1; x2max = 6;

x1step = .25; x2step = .25;

[x1, x2] = meshgrid(x1min:x1step:x1max, x2min:x2step:x2max);

a = 1.5;
b = 1.1;
c = 2.5;
d = 1.4;

dx1 = -a.*x1 + b.*x1.*x2;
dx2 = c.*x2 - d.*x1.*x2;

dx1 = dx1./sqrt(dx1.^2 + dx2.^2);
dx2 = dx2./sqrt(dx1.^2 + dx2.^2);

quiver(x1, x2, dx1,dx2,'AutoScaleFactor',0.5)

axis([x1min x1max x2min x2max])

xlabel('Predator Population (dozens)','Interpreter','latex')
ylabel('Prey Population (dozens)','Interpreter','latex')
title('Lotka-Volterra System','Interpreter','latex')

[t_out, v_out] = ode45(@solutions,[0,20],[.5,1]);
plot(v_out(:,1), v_out(:,2))
```

Plotting Component Curves of The Lotka-Volterra System (See Function 6)

```
[t_out, v_out] = ode45(@solutions,[0,20],[.5,1]);

hold on
plot(t_out, v_out(:,1))
plot(t_out, v_out(:,2))

legend('Predator Population','Prey Population')
title('Lotka-Volterra System Over Time')
xlabel('Time (days)')
ylabel('Population (dozens)')
```

Matlab Code for Section 3.2

Plotting Phase Portrait, Nullclines, and Phase Plane Solution (Modified flow.m, See Function 7)

```
x1min = -1; x1max = 6; x2min = -1; x2max = 6;

x1step = .25; x2step = .25;

[x1, x2] = meshgrid(x1min:x1step:x1max, x2min:x2step:x2max);
```

```
a = 1.5;
b = 1.1;
c = 2.5;
d = 1.4;
k = 0.5;

dx1 = -a*x1 + b*x1.*x2;
dx2 = c*(1-k*x2).*x2 - d*x1.*x2;

dx1 = dx1./sqrt(dx1.^2 + dx2.^2);
dx2 = dx2./sqrt(dx1.^2 + dx2.^2);

quiver(x1, x2, dx1,dx2,'AutoScaleFactor',0.5)

axis([x1min x1max x2min x2max])

xlabel('Predator Population (dozens)','Interpreter','latex')
ylabel('Prey Population (dozens)','Interpreter','latex')
title('Logistic Predator-Prey System','Interpreter','latex')

hold on
[t,v] = ode45(@logistic_solver,[0,50],[0.5,1]);
plot(v(:,1),v(:,2),'LineWidth',2);
xline(0,'k');
yline((a/b),'k');
yline(0,'r');
x1 = -1:0.1:6;
x2 = (1/k).*(1-(d/c).*x1);
plot(x1,x2,'r');
plot(0,0,'g*','Linewidth',5);
plot(((c/d)*(1-k*(a/b))),(a/b),'g*','LineWidth',5);
plot(0,(1/k),'g*','Linewidth',5);
hold off
```

Plotting the Component Curves (See Function 7)

```
figure
hold on
grid on
[t_out,v_out] = ode45(@logistic_solver,[0,50],[0.5,1]);
plot(t_out,v_out(:,1))
plot(t_out,v_out(:,2))
xlabel('Time (days)');
ylabel('Population (dozens)');
title('Logistic Predator-Prey Component Curves');
legend('x_1(t) - Predator','x_2(t) - Prey');
hold off
```